



MACHINE LEARNING · 3RD-YEAR SOFTWARE ENGINEERING

Introduction to Machine Learning

Building the shared vocabulary and mental model for the rest of the course.

Dr. OM HENI Nizar



From writing rules to learning them

Most software you have written is rule-based: you specify the logic, the program executes it. Machine Learning inverts this — you supply examples and an algorithm infers the rules.



Rule-based program

- Engineer writes the conditions by hand
- Breaks when rules are too many or change too often
- Some rules nobody can even articulate
- e.g. spam filter maintained forever, by hand



Learned model (ML)

- Trained on thousands of labeled examples
- Infers the patterns from data itself
- When spam evolves, you retrain — not rewrite
- A component whose behavior is learned, not coded

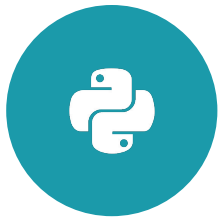
For a Software Engineer: a new kind of component — testable, reproducible, monitored.



What you will be able to do

- 1 Define & locate ML** Define Machine Learning and place it relative to AI and Deep Learning.
- 2 Distinguish the paradigms** Tell apart supervised, unsupervised, semi-supervised, and reinforcement learning — with examples.
- 3 Describe the workflow** Walk the end-to-end ML workflow and the distinct roles of training, validation, and test data.
- 4 Reason about generalization** Explain why performance must be measured on data the model has never seen.
- 5 Compute a baseline** Load a dataset, identify features and target, and compute a baseline in Python.

What you should already have



Programming

- Python: functions, modules
- Basic NumPy / Pandas familiarity



Mathematics

- The idea of a function $f(x) = y$
- Basic probability & statistics (mean)



Software engineering

- Separation of concerns
- Here: separating data from logic

No prior ML knowledge is assumed — everything else is built from here.

Learning = Task · Performance · Experience

Mitchell's classic formulation: a program learns if its performance on a task T, as measured by P, improves with more experience E.



T — Task

What the program does

Spam filter: Classifying emails as spam / not-spam



P — Performance

How we measure success

Spam filter: Classification accuracy (or a better metric)

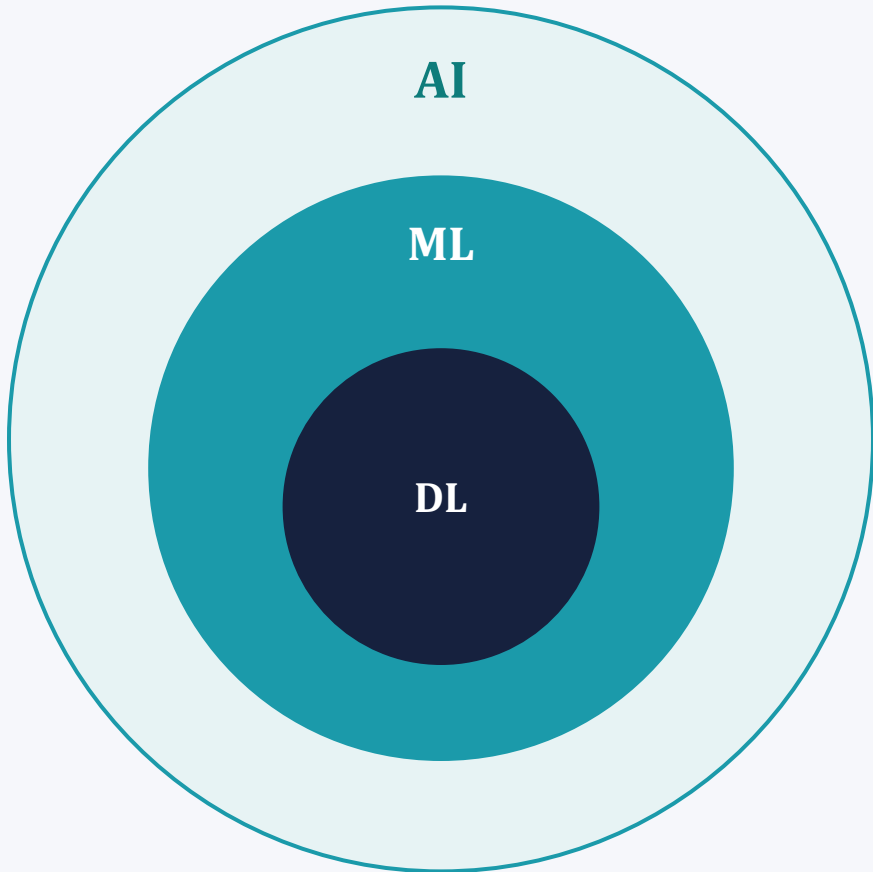


E — Experience

What it learns from

Spam filter: The labeled emails it trains on

$AI \supseteq ML \supseteq \text{Deep Learning}$



● Artificial Intelligence

Systems performing tasks we associate with intelligence — rule-based systems, search, planning, and ML.

● Machine Learning

The subset of AI where behavior is learned from data rather than hand-coded.

● Deep Learning

A subset of ML using multi-layer neural networks — strong on images, audio, language, and large data.

All DL is ML • not all ML is DL • not all AI is ML.

Kinds of learning



Supervised

Main focus · Ch. 3–8

Data has labels; learn a mapping $x \rightarrow y$.

Spam detection, house-price prediction



Unsupervised

Ch. 9–10

No labels; find structure in the data itself.

Customer segmentation, dimensionality reduction

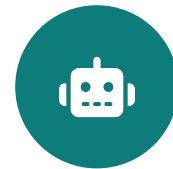


Semi-supervised

Perspective

A little labeled data + lots of unlabeled.

Used when labeling is expensive



Reinforcement

Brief perspective

An agent acts and receives rewards.

Game playing, robotics



The words we will use all course long

Sample / instance one data point, (x_i, y_i) .

Feature a measurable property; x_i is a vector of d features.

Label / target the value to predict, y_i .

Dataset the full collection $D = \{(x_i, y_i)\}$.

Model / hypothesis a function f_θ with parameters θ .

Hypothesis space $\mathcal{H}$ the set of candidate functions to choose from.

Learning algorithm searches $\mathcal{H}$ for parameters θ that fit D well.

Supervised goal

$$f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$$

find f_θ that predicts well on new, unseen inputs from the same source.

Classification vs Regression

The target's type decides the task: categorical → classification, continuous → regression.



Classification

y is categorical

- Predict a class / discrete category
- Spam vs not-spam; flower species
- Iris: 3 species from 4 measurements



Regression

y is continuous

- Predict a number on a continuous scale
- House price; tomorrow's temperature
- Output lives in a range, not a set

Loss, empirical risk, expected risk

Loss $L(y, \hat{y})$

how wrong one prediction $\hat{y} = f_{\theta}(x)$ is versus the truth y . Example (regression): squared error

$$L(y, \hat{y}) = (y - \hat{y})^2$$

Empirical risk

average loss on the training data

$$R_{\text{emp}}(f_{\theta}) = \frac{1}{n} \sum_{i=1}^n L(y_i, f_{\theta}(x_i))$$

Training a model = choose θ to make it small $\rightarrow$ empirical risk minimization.

Expected (true) risk

average loss over the whole distribution $\mathcal{P}$

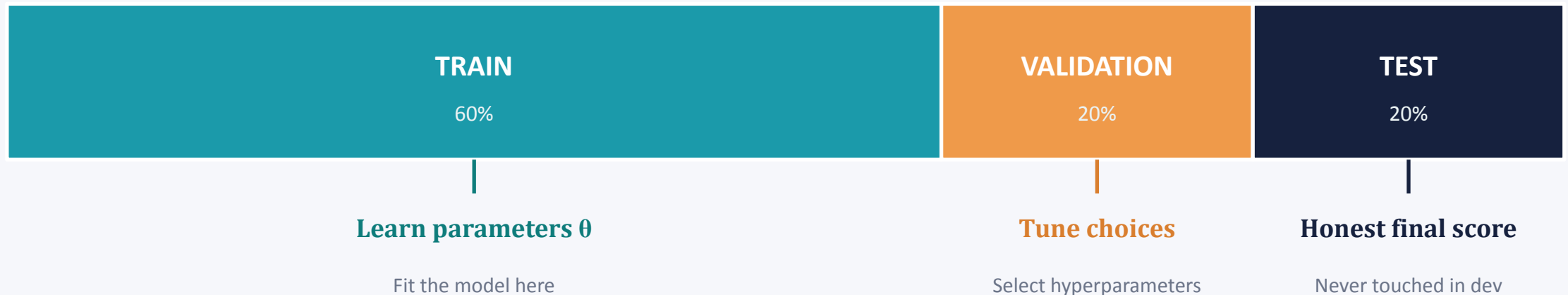
$$R(f_{\theta}) = \mathbb{E}_{(x,y) \sim \mathcal{P}}[L(y, f_{\theta}(x))]$$

Cannot be computed — we estimate it on data never trained on: the test set.

Low training error does not guarantee low true error — that gap is generalization, overfitting, underfitting.

The train / validation / test split

Because true risk must be estimated on unseen data, we split the dataset into three disjoint parts.



The test set is the final exam: it must stay untouched during development, or its estimate is no longer honest.

Learning is like preparing for an exam



Training

Studying past problems with worked answers.



Validation

Practice tests to decide how to study (tuning).



The final exam

Unseen questions — did you understand or memorize?

Overfitting

Memorizes past answers verbatim → aces the homework, fails the exam.

Underfitting

Barely studied → fails the homework and the exam alike.

Good learning generalizes — from worked examples to new situations it has never seen.



Baselines — the bar every model must clear

Majority-class baseline

classification

Always predict the most frequent class in the training data.

Mean baseline

regression

Always predict the training-set mean of y .

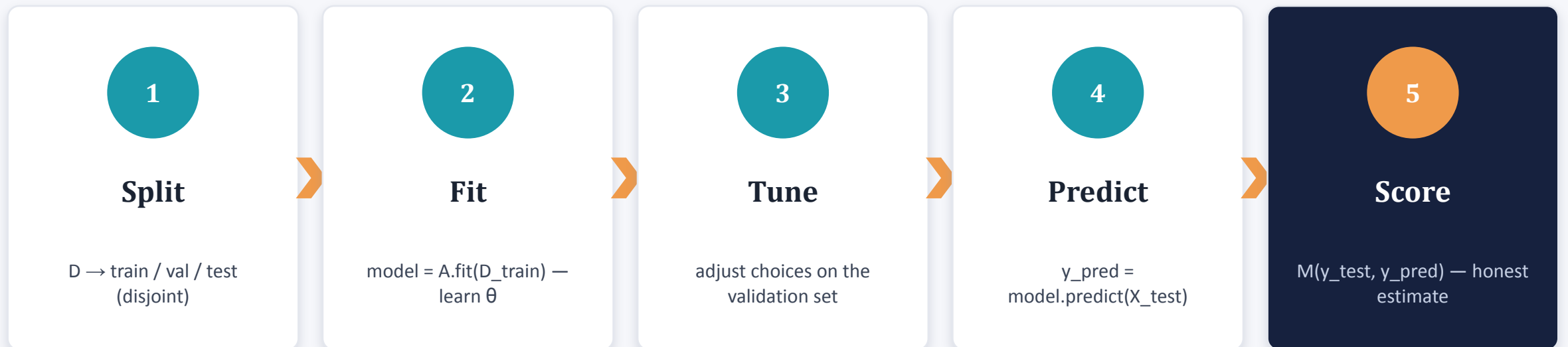
Majority-class baseline — pseudocode

```
FIT(D_train):  
    c* <- most frequent label in y_train  
    store c*  
PREDICT(X):  
    return c* for every row in X
```

Why bother?

A baseline is the reference that turns “92% accuracy” into meaningful — or embarrassing. It is not a toy; every later model must beat it.

The supervised procedure, end to end



Input: dataset D , algorithm A , metric M . Output: a trained model and an honest performance score.

Iris — 150 samples, 4 features, 3 species

1

Identify structure

4 measurements (sepal / petal length & width) → species (3 classes).

2

Split

Hold out ~30% as a test set the model never sees while fitting.

3

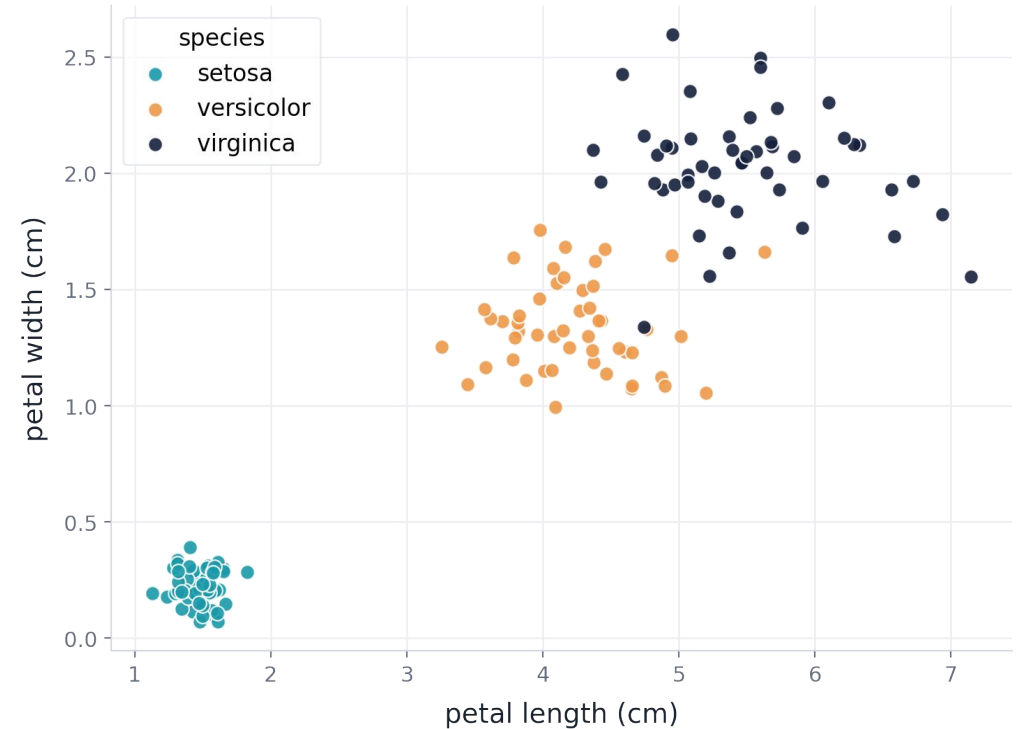
Baseline

Majority class → about one-third accuracy (3 balanced classes).

4

Interpretation

~0.33 means it learned nothing; ~0.95 means the 4 measurements truly separate species.

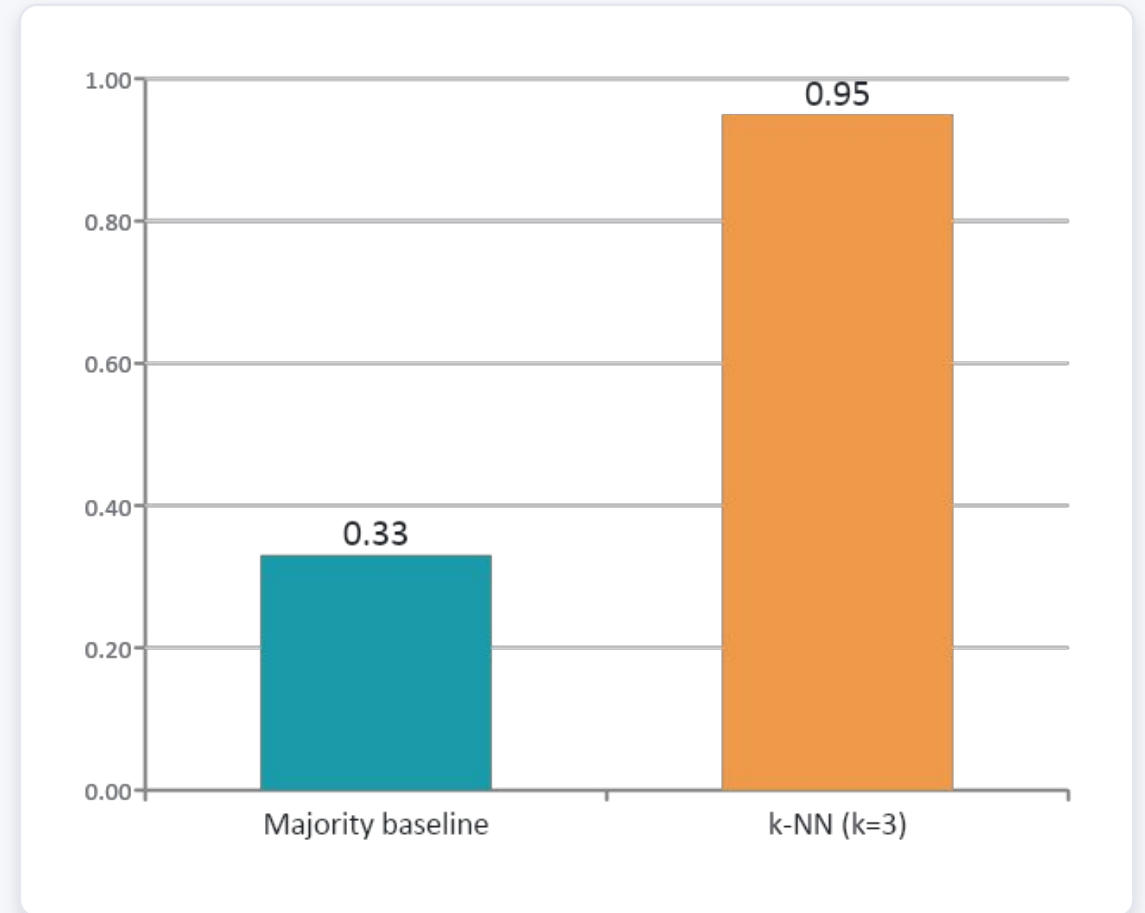


Classes form visibly distinct regions — that separability is what a classifier exploits.

Making an accuracy number mean something

On Iris, the majority-class baseline sits near the floor; a simple k-NN clears it decisively. The gap is the evidence of learning.

- Baseline $\approx$ one-third — predicts the majority species for everything
- k-NN ($k = 3$) ≈ 0.95 — genuinely uses the four measurements
- Without the baseline, “0.95” would be a number with no reference



Illustrative test-set accuracies (approximate).

The baseline — by hand, then with scikit-learn

NumPy — expose the logic

```
X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.30,
    random_state=42, stratify=y)

# majority-class baseline, explicit
vals, cnts = np.unique(
    y_tr, return_counts=True)
majority = vals[np.argmax(cnts)]
pred = np.full(y_te.shape, majority)
acc = np.mean(pred == y_te)
```

scikit-learn — same result

```
from sklearn.dummy import DummyClassifier
from sklearn.neighbors import (
    KNeighborsClassifier)

dummy = DummyClassifier(
    strategy="most_frequent"
).fit(X_tr, y_tr)
knn = KNeighborsClassifier(
    n_neighbors=3).fit(X_tr, y_tr)

dummy.score(X_te, y_te) # baseline
knn.score(X_te, y_te)  # first model
```

Every estimator shares one contract: `.fit(X, y)` learn · `.predict(X)` infer · `.score(X, y)` default metric.

A model is a component in a larger system



Reproducibility from day one

Fix random seeds (`random_state`). A result you cannot reproduce is not yet an engineering result.



Separation of concerns

Keep data loading, preprocessing, modeling, and evaluation in distinct units — not one giant cell.



The model is one component

It sits behind data validation, a pipeline, and an API — a versioned, testable artifact.



Data is part of the system

Behavior depends on training data, so the data must be versioned and documented too.

Common mistakes & failure modes



Common mistakes

- Evaluating on training data — the #1 beginner error
- No baseline — “90%” may already be trivial on imbalanced data
- Accuracy on imbalanced classes — 95% “not spam” catches zero spam
- Data leakage — test info leaking into training (e.g. scaling before splitting)



Failure modes

- Garbage in, garbage out — no algorithm rescues bad data
- Distribution shift — the world changes; $\mathcal{P}$ no longer matches production
- Metric–goal mismatch — optimizing accuracy when rare positives are what matter

Assumptions & how to read a prediction



i.i.d. & representative

Training and future data are drawn independently from the same distribution $\mathcal{P}$.



Stationarity

$\mathcal{P}$ is (approximately) stable over the timeframe the model is used.



Learnable signal

There is a real relationship between features and target; no model invents signal.



Interpretation

A model's output is a prediction — not a fact, and usually not a cause.

A churn model ranking “support tickets” as important has found correlation, not proof of cause. Communicate predictions with their uncertainty; resist reading causation into association.

If production data differs from training data, the guarantees evaporate — assumptions are not fine print.

Chapter 1 in six lines

- 1 ML learns behavior from data instead of encoding it as explicit rules.
- 2 $AI \supseteq ML \supseteq \text{Deep Learning}$; paradigms: supervised, unsupervised, semi-supervised, reinforcement.
- 3 The target is generalization to unseen data — not memorization of training data.
- 4 Always hold out a test set, and always beat a baseline.
- 5 Reproducibility, separation of concerns, and data versioning are requirements, not extras.
- 6 Assumptions (i.i.d., representative, stationary, learnable signal) underlie everything that follows.

Exercises

1

Define in one sentence each: feature, label, model, hypothesis space.

2

Classify as supervised / unsupervised / reinforcement: (a) predicting tomorrow's temperature; (b) grouping shoppers with no labels; (c) a robot learning to walk via reward.

3

For a dataset where 80% of samples are class A, compute the majority-class baseline accuracy.

4

Explain, in your own words, why test accuracy can be lower than training accuracy.

5

Name a real software feature (likely) powered by ML and state its plausible T, P, E.

Advanced — the accuracy trap: a fraud detector labels 1% as fraud. A model predicts “not fraud” for everything. Compute its accuracy and propose a better metric.

Foundational sources

- **James, Witten, Hastie, Tibshirani** — *An Introduction to Statistical Learning (Springer)* — a Python edition (ISLP) also exists.
- **Hastie, Tibshirani, Friedman** — *The Elements of Statistical Learning (Springer)*.
- **C. M. Bishop** — *Pattern Recognition and Machine Learning (Springer)*.
- **A. Géron** — *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (O'Reilly)*.
- **T. M. Mitchell** — *Machine Learning (McGraw-Hill)* — source of the task / performance / experience definition.
- **Pedregosa et al. (2011)** — “Scikit-learn: Machine Learning in Python,” JMLR — the library used throughout.

Editions, years, and identifiers to be confirmed in Phase 7 — no page numbers, DOIs, or URLs are asserted here.

End of Chapter 1 — next: the ML workflow & data (Chapter 2).